

A Project Report
On
FAKE NEWS DETECTION USING NLP

(Web-Based Application)

*Submitted in partial fulfilment of the
requirement for the award of the degree of*

MASTER OF COMPUTER APPLICATION

Session 2025-26
in
Computer Application

By

Shalini Kumari
23SCSE1040450

Palak Shrivastav
23SCSE1040119

Akarsh Yadav
23SCSE1040160

Under the guidance of
Mr Salil Kumar Srivastava

SCHOOL OF COMPUTER APPLICATIONS AND TECHNOLOGY
GALGOTIAS UNIVERSITY, GREATER NOIDA
INDIA

June, 2026

**SCHOOL OF COMPUTER APPLICATIONS AND TECHNOLOGY
GALGOTIAS UNIVERSITY, GREATER NOIDA**

CANDIDATE'S DECLARATION

We hereby certify that the work which is being presented in the project, entitled "Fake News Detection Using NLP" in partial fulfilment of the requirements for the award of the BCA (Bachelor's of Computer Application) submitted in the School of Computer Applications and Technology of Galgotias University, Greater Noida, is an original work carried out during the period of August 2025 to January 2026, under the supervision of Mr Salil Kumar Srivastava, Department of School of Computer Applications and Technology, Galgotias University, Greater Noida.

The matter presented in the project has not been submitted by us for the award of any other degree of this or any other place.

Student's Name:-

Shalini Kumari
23SCSE1040450

Palak Shrivastava
23SCSE1040119

--

Akarsh Yadav
23SCSE1040160

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

Guide Name:-

Mr Salil Kumar Srivastava

Designation:-

Assistant Professor

**SCHOOL OF COMPUTER APPLICATIONS AND TECHNOLOGY
GALGOTIAS UNIVERSITY, GREATER NOIDA**

CERTIFICATE

This is to certify that the Project Report entitled "Fake News Detection Using NLP" which is submitted by **Shalini Kumari, Palak Shrivastav, and Akarsh Yadav** in partial fulfilment of the requirement for the award of degree BCA in Department of Computer Application of School of Computer Applications and Technology, Galgotias University, Greater Noida, India is a record of the candidate's own work carried out by them under my supervision. The matter embodied in this project is original and has not been submitted for the award of any other degree.

Signature of Examiner(s)

Signature of Supervisor(s)

Date: 11 June 2026

Place: Greater Noida

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our project guide **Mr Salil Kumar Srivastava, Assistant Professor** of School of Computer Applications and Technology, Galgotias University, for their valuable guidance, constant encouragement, and constructive suggestions throughout the course of this project.

We are also thankful to the Head of Department and all faculty members of the School of Computer Applications and Technology for their support and cooperation. We extend our appreciation to our classmates and friends for their moral support during this work.

Finally, We would like to acknowledge the academic resources and datasets that were instrumental in completing this project on Fake News Detection Using Natural Language Processing.

Student Name(s)

Shalini Kumari
23SCSE1040450

Palak Shrivastav
23SCSE1040119

Akarsh Yadav
23SCSE1040160

ABSTRACT

The rapid proliferation of digital content on social media and online news platforms has created an urgent need for automated systems capable of distinguishing genuine news from fabricated or misleading information. Fake news has emerged as a critical societal challenge, capable of influencing public opinion, triggering panic, and undermining democratic processes.

This project presents the design and development of a web-based Fake News Detection system powered by Natural Language Processing (NLP) techniques. The system enables users to input news headlines or full article text and receive an instant classification of whether the content is likely to be real or fake.

The backend employs NLP preprocessing pipelines including tokenization, stop-word removal, and TF-IDF vectorization, combined with machine learning classifiers such as Logistic Regression and Passive Aggressive Classifier trained on publicly available labeled datasets. The frontend is built as a responsive web application providing an intuitive user interface for seamless interaction.

Experimental results demonstrate that the proposed system achieves high accuracy in detecting fake news, making it a practical tool for journalists, researchers, and the general public.

Keywords: Fake News Detection, Natural Language Processing, TF-IDF, Machine Learning, Web Application, Text Classification

TABLE OF CONTENTS

CANDIDATE'S DECLARATION.....	2
CERTIFICATE.....	3
ACKNOWLEDGEMENTS.....	4
ABSTRACT.....	5
TABLE OF CONTENTS.....	6
LIST OF TABLES.....	8
LIST OF FIGURES	8
LIST OF ABBREVIATIONS.....	8
CHAPTER 1	10
INTRODUCTION	10
1.1 Problem Introduction	10
1.2 Motivation.....	10
1.3 Project Objective.....	10
1.4 Scope of the Project	11
1.5 Related Previous Work	12
1.6 Organization of the Report.....	12
CHAPTER 2	13
SOFTWARE REQUIREMENT SPECIFICATION (SRS).....	13
2.1 Product Perspective.....	13
2.1.1 System Interfaces	13
2.1.2 User Interfaces	13
2.1.3 Hardware Interfaces	13
2.1.4 Software Interfaces	13
2.1.5 Communications Interfaces	14
2.1.6 Memory Constraints.....	14
2.1.7 Operations	14
2.1.8 Site Adaptation Requirements	14
2.2 Product Functions	14
2.3 User Characteristics	15
2.4 Constraints	15
2.5 Assumptions and Dependencies	15

2.6 Apportioning of Requirements	15
2.7 Use Case Model	16
2.7.1 Actors	16
2.7.2 Use Case Diagram.....	16
2.7.3 Use Case Scenario.....	16
2.8 Sequence Diagram	17
CHAPTER 3	18
SYSTEM DESIGN	18
3.1 Architecture Diagram.....	18
3.2 Data Flow Diagram.....	19
3.2.1 DFD Level 0 — Context Diagram.....	19
3.2.2 DFD Level 1	19
3.3 Activity Diagram	20
3.4 ER Diagram	21
3.5 Database Schema	22
CHAPTER 4	23
IMPLEMENTATION AND RESULTS.....	23
4.1 Software and Hardware Requirements	23
4.2 Implementation Details.....	23
4.2.1 NLP Preprocessing Pipeline	23
4.2.2 Model Training	24
4.3 Constraints	24
4.4 Snapshots of Interfaces	25
4.5 Test Cases	25
4.6 Results.....	25
CHAPTER 5	29
CONCLUSION.....	29
5.1 Performance Evaluation.....	29
5.2 Comparison with Existing State-of-the-Art Technologies	29
5.3 Future Directions	29
APPENDIX.....	30
REFERENCES	29

LIST OF TABLES

Table No.	Title	Page No.
4.1	Hardware Requirements	xx
4.2	Software Requirements	xx
4.3	Model Evaluation Results	xx
4.4	Test Cases for News Classification	xx

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	3-Tier Architecture Diagram	xx
3.2	Use Case Diagram	xx
3.3	Data Flow Diagram — Level 0	xx
3.4	Data Flow Diagram — Level 1	xx
3.5	Activity Diagram	xx
3.6	ER Diagram	xx
4.1	NLP Processing Pipeline	xx
4.2	Model Performance Comparison Chart	xx
4.3	Confusion Matrix — Logistic Regression	xx

LIST OF ABBREVIATIONS

Abbreviation	Full Form
NLP	Natural Language Processing
ML	Machine Learning

TF-IDF	Term Frequency-Inverse Document Frequency
SRS	Software Requirement Specification
API	Application Programming Interface
UI	User Interface
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
GUI	Graphical User Interface
HTTP	HyperText Transfer Protocol
PAC	Passive Aggressive Classifier
LR	Logistic Regression
ER	Entity-Relationship
DFD	Data Flow Diagram

CHAPTER 1

INTRODUCTION

In the digital era, information spreads at an unprecedented pace through social media platforms, blogs, and online news portals. While this democratization of information has numerous benefits, it has also given rise to a serious threat — the propagation of fake news. Fake news refers to deliberately fabricated or misleading content presented as legitimate news, often designed to manipulate public opinion, incite conflict, or generate engagement through sensationalism.

Natural Language Processing (NLP), a branch of Artificial Intelligence, provides powerful tools to analyze, understand, and process human language. By applying NLP techniques to news text, it is possible to build automated systems that can classify news as real or fake with reasonable accuracy.

This project, "Fake News Detection Using NLP," aims to develop a web-based application that uses NLP and Machine Learning to detect fake news in real time. Users can paste any news content or headline into the application and receive an immediate classification verdict.

1.1 Problem Introduction

The spread of misinformation through online channels has become a major challenge for governments, news organizations, and the general public alike. Traditional manual fact-checking methods are time-consuming and cannot scale to the volume of content published daily. There is an urgent need for automated, intelligent tools that can assist in identifying potentially fake or misleading news.

1.2 Motivation

The increasing influence of fake news on elections, public health, and social harmony motivated the development of this project. The COVID-19 pandemic witnessed a parallel "infodemic" — a flood of misinformation that negatively impacted public response. NLP and machine learning offer scalable, data-driven solutions to combat this problem effectively.

1.3 Project Objective

The primary objectives of this project are:

- To build a web-based application for real-time fake news detection.
- To apply NLP preprocessing techniques such as tokenization, stop-word removal, and TF-IDF vectorization.
- To train and evaluate machine learning classifiers (Logistic Regression, Passive Aggressive Classifier) on labeled datasets.
- To provide an intuitive user interface for easy interaction.
- To achieve high classification accuracy on standard benchmark datasets.

1.4 Scope of the Project

The system is designed to:

- Accept news text or headlines as input from the user via a web browser.
- Process the input through an NLP pipeline and return a real/fake classification.
- Work with English-language news content.
- Be accessible across devices through a responsive web interface.

The system does not fact-check against live databases of verified news, but relies on learned patterns from training data.

1.5 Role of Natural Language Processing (NLP)

Natural Language Processing (NLP) is a branch of Artificial Intelligence that helps computers understand and process human language. Since news articles are text-based, NLP techniques are used to analyze the content and identify patterns in fake and real news.

In this project, NLP techniques such as tokenization, stop-word removal, lowercasing, and TF-IDF vectorization are used to preprocess the news text before classification. These techniques improve the accuracy and efficiency of the machine learning model.

1.6 Challenges in Fake News Detection

Fake news detection is a difficult task because fake articles are often written in a way that looks similar to real news. Social media platforms spread information very quickly, making manual verification challenging.

Other challenges include misleading headlines, sarcasm, biased datasets, and multilingual content. These factors can affect the accuracy of machine learning models and make fake news detection more complex.

1.7 Advantages of the Proposed System

The proposed system provides fast and automated fake news detection using NLP and Machine Learning techniques. Users can enter news text into the web application and instantly receive prediction results.

The system is user-friendly, efficient, and capable of handling large amounts of textual data. It also reduces manual effort and can be further improved in the future with advanced AI models and multilingual support.

1.8 Related Previous Work

Significant research has been conducted in the area of fake news detection. Shu et al. (2017) proposed a comprehensive survey of fake news detection methods, categorizing approaches into knowledge-based, style-based, propagation-based, and source-based methods. Riedel et al. (2017) introduced the Fake News Challenge dataset. Ahmed et al. (2017) demonstrated that TF-IDF combined with Logistic Regression achieves over 90% accuracy on standard datasets. More recent work explores deep learning models such as BERT for improved contextual understanding of news content.

1.9 Organization of the Report

The rest of this report is organized as follows:

- Chapter 2 presents the Software Requirement Specification (SRS) for the web-based application.
- Chapter 3 covers the System Design, including architecture, UML diagrams, and database schema.
- Chapter 4 describes the Implementation details and presents the results.
- Chapter 5 concludes the report and outlines future directions.

CHAPTER 2

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

This chapter describes the Software Requirement Specification for the Fake News Detection web application. It outlines the product perspective, functions, user characteristics, constraints, and use case models that define the scope and behavior of the system.

2.1 Product Perspective

The Fake News Detection system is a standalone web application that operates independently. It accepts user-provided news text and returns a prediction. The system interacts with a trained machine learning model deployed on the server side.

2.1.1 System Interfaces

The system interfaces with: (1) User browser — for submitting news text and displaying results; (2) Backend ML model — a serialized Scikit-learn model loaded at server startup; (3) Dataset files — CSV files containing labeled news data used during training.

2.1.2 User Interfaces

The application provides a clean, responsive web-based GUI built with HTML, CSS, and JavaScript. The main interface consists of a text input area, a submit button, and a result display panel. The UI is designed to be accessible and usable across desktops, tablets, and mobile devices.

2.1.3 Hardware Interfaces

The system has no specialized hardware interface requirements. It runs on any standard server or local machine with sufficient CPU and RAM.

2.1.4 Software Interfaces

The system uses the following software components:

- Python 3.9+ — Core programming language
- Flask 2.x — Lightweight web framework for the backend
- Scikit-learn 1.x — Machine learning library for model training and inference

- NLTK / spaCy — NLP preprocessing libraries
- Pandas / NumPy — Data manipulation
- HTML5 / CSS3 / JavaScript — Frontend technologies
- Bootstrap 5 — Responsive UI framework

2.1.5 Communications Interfaces

The system communicates over HTTP/HTTPS protocols. The client sends POST requests with news text to the server REST API endpoint, and the server responds with JSON containing the classification result and confidence score.

2.1.6 Memory Constraints

The server should have a minimum of 2 GB RAM to load the ML model and handle concurrent requests. The trained model file is expected to be under 50 MB.

2.1.7 Operations

Normal operations include users submitting news text and receiving predictions. The system operates in interactive mode at all times. Backup operations include periodic saving of model files and logs.

2.1.8 Site Adaptation Requirements

For deployment on a remote server, the trained model file (.pkl) and web application files must be uploaded. Python dependencies listed in requirements.txt must be installed.

2.2 Product Functions

- Text Input: Accept news article text or headline from the user.
- NLP Preprocessing: Tokenize, clean, and vectorize the input text using TF-IDF.
- News Classification: Apply the trained ML model to classify the news as Real or Fake.
- Result Display: Show the classification result along with a confidence percentage.
- History (Optional): Allow users to view past queries within a session.

2.3 Non-Functional Requirements

The system should provide high accuracy and fast response time during prediction. The web application must be reliable, secure, and easy to use across different devices and browsers.

The application should also support scalability so that future features such as multilingual support and advanced AI models can be integrated easily.

2.4 User Characteristics

Intended users: (1) General public — basic computer literacy, web browser access; (2) Journalists and fact-checkers — quick preliminary screening; (3) Researchers — studying NLP misinformation detection. The UI accommodates all levels of technical expertise.

2.5 Constraints

- Limited to English-language news in the current version.
- Classification accuracy depends on training dataset quality and size.
- No verification against real-time fact-checking databases.
- Server uptime depends on hosting environment.
- Model may exhibit bias if training data is unbalanced.

2.5 Security Requirements

The system should validate user input before processing to avoid invalid or harmful data. Secure communication protocols such as HTTPS should be used during deployment to protect user information.

Only authorized administrators should have access to system files, datasets, and trained machine learning models.

2.6 Assumptions and Dependencies

- Users provide complete and meaningful news text (not single words).
- Scikit-learn and NLTK libraries are properly installed.
- A modern web browser (Chrome, Firefox, Edge) is available on the client side.
- The training dataset is pre-labeled with reliable ground truth.

2.7 Apportioning of Requirements

Phase 1 (Current): Core news classification, web interface, TF-IDF + Logistic Regression.

Phase 2 (Future): BERT integration, multilingual support, user accounts, history tracking.

Phase 3 (Future): Browser extension, third-party API, real-time news feed scanning.

2.8 Use Case Model

2.8.1 Actors

- User — Submits news text and views classification result.
- System — Web application and its NLP/ML backend.

2.8.2 Use Case Diagram

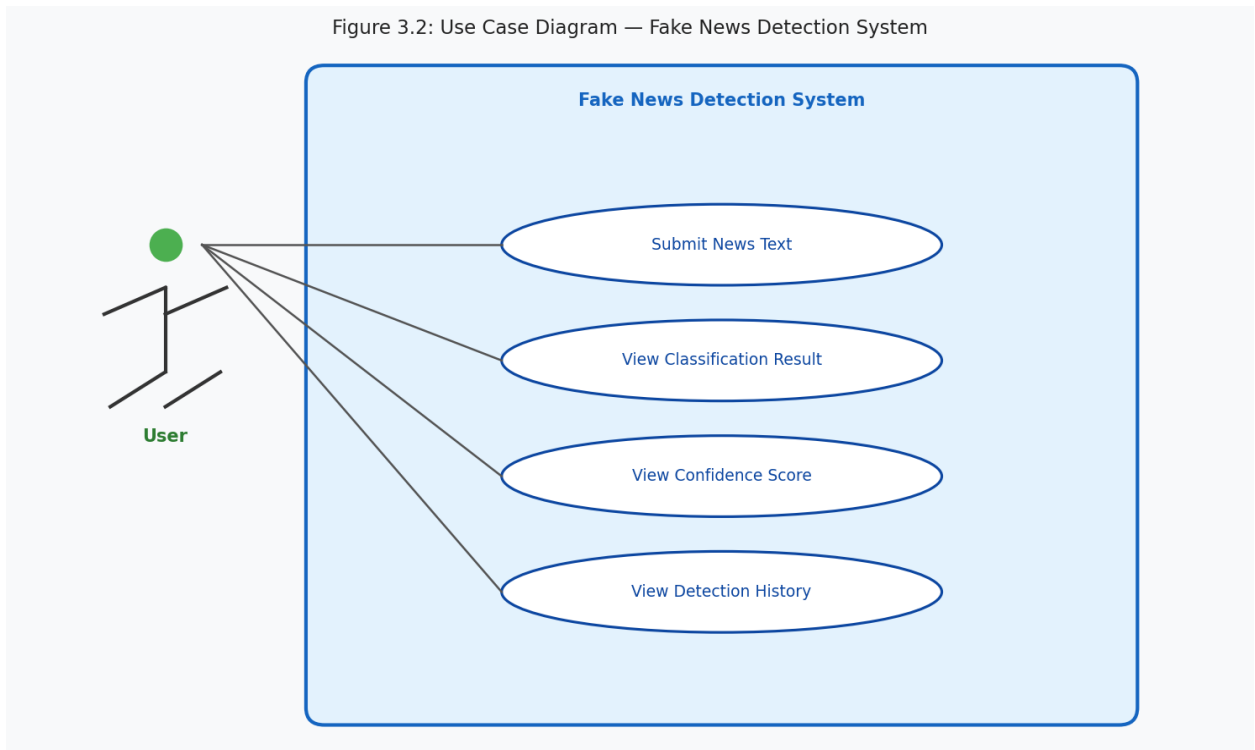


Figure 3.2: Use Case Diagram — Fake News Detection System

2.8.3 Use Case Scenario

Use Case Number	UC-01
Application	Fake News Detection Web Application
Use Case Name	Detect Fake News
Use Case Description	User submits news text/headline; system classifies it as Real or Fake.
Primary Actor	User

Precondition	User has access to the web application and a stable internet connection.
Trigger	User enters news text and clicks the 'Detect' button.
Basic Flow	1. User opens web app. 2. User enters news text. 3. User clicks Detect. 4. System preprocesses text. 5. System applies ML model. 6. System shows REAL/FAKE with confidence.
Alternate Flows	A1: Empty input — validation error shown. A2: Server error — generic error message displayed.

2.9 Sequence Diagram

The sequence diagram shows the interaction between: User → Web Browser → Flask Server → NLP Module → ML Model → Result. The user submits text; the browser POSTs to Flask; Flask preprocesses via NLP module; the ML model returns a prediction; Flask responds with JSON; the browser renders the result.

CHAPTER 3

SYSTEM DESIGN

This chapter presents the complete system design of the Fake News Detection web application, including system architecture, use case diagram, data flow diagrams, activity diagram, ER diagram, and database schema.

3.1 Architecture Diagram

The system follows a 3-Tier Architecture: Presentation Layer (HTML/CSS/JS frontend), Application Logic Layer (Flask server + NLP + ML model), and Data Layer (datasets, model file, SQLite).

Figure 3.1: 3-Tier Architecture Diagram — Fake News Detection System

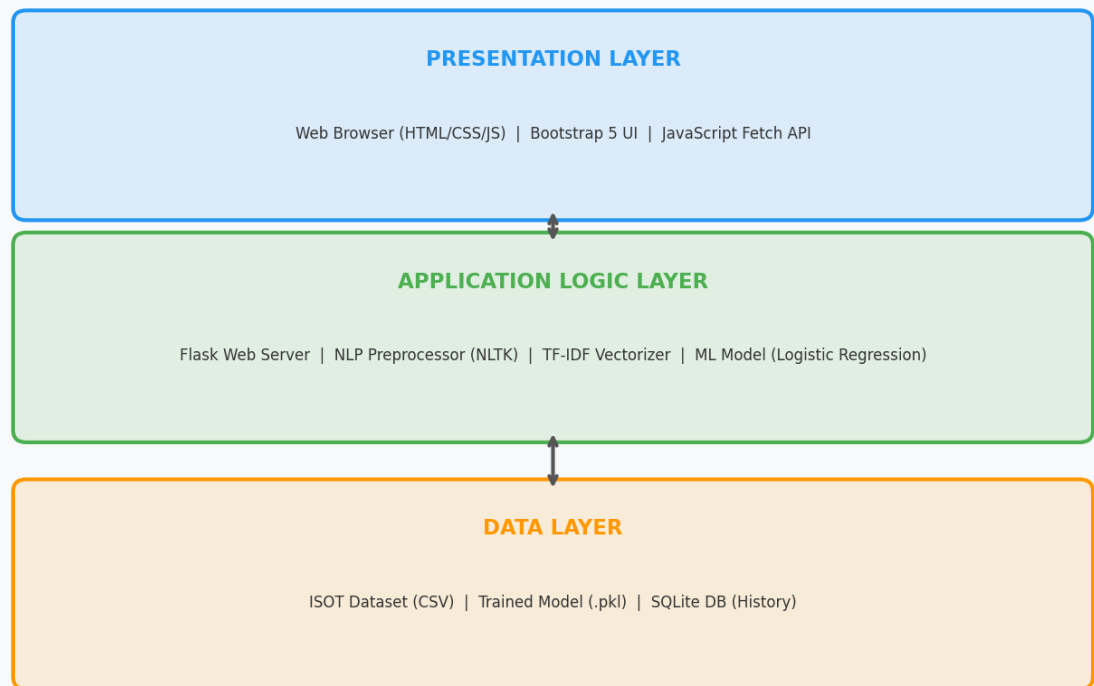


Figure 3.1: 3-Tier Architecture Diagram — Fake News Detection System

3.1.1 System Workflow

The workflow of the system begins when the user enters news text into the web application. The input text is sent to the Flask backend server, where NLP preprocessing techniques are applied.

After preprocessing, the text is converted into numerical vectors using TF-IDF vectorization. The trained machine learning model then predicts whether the news is real or fake, and the result is displayed to the user.

3.2 Data Flow Diagram

3.2.1 DFD Level 0 — Context Diagram

At Level 0, the system is a single process. The external entity 'User' provides News Text as input and receives a Classification Result as output.

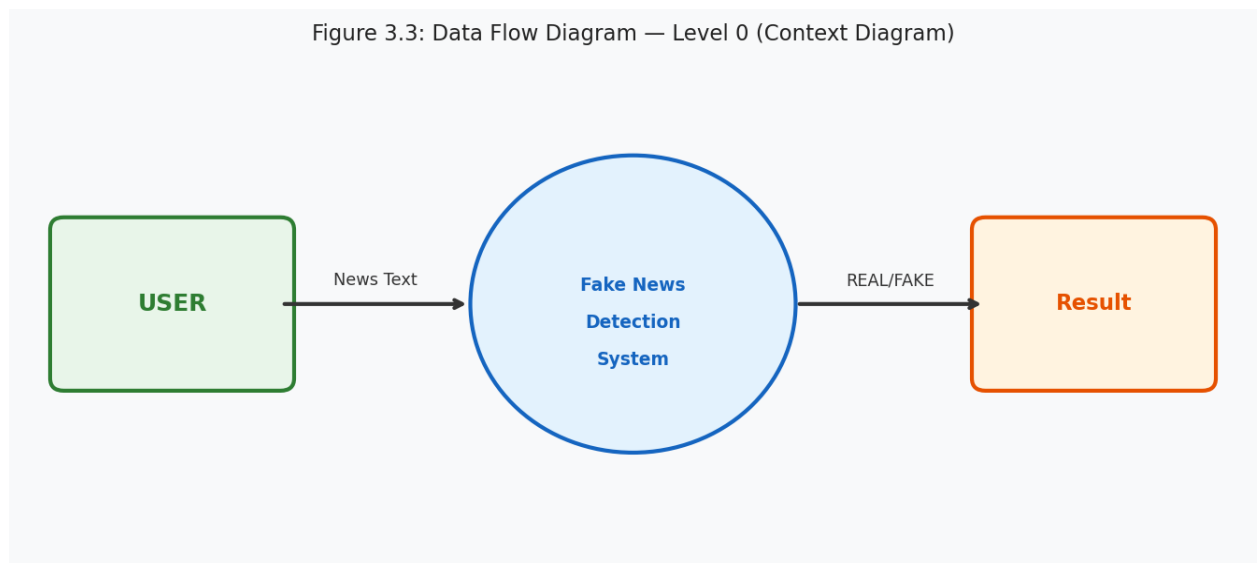


Figure 3.3: Data Flow Diagram — Level 0 (Context Diagram)

3.2.2 DFD Level 1

Level 1 decomposes the system into five processes: Input Handling, NLP Preprocessing, TF-IDF Vectorization, ML Model Prediction, and Result Display. Data stores include the Model Store (.pkl) and Vectorizer Store.

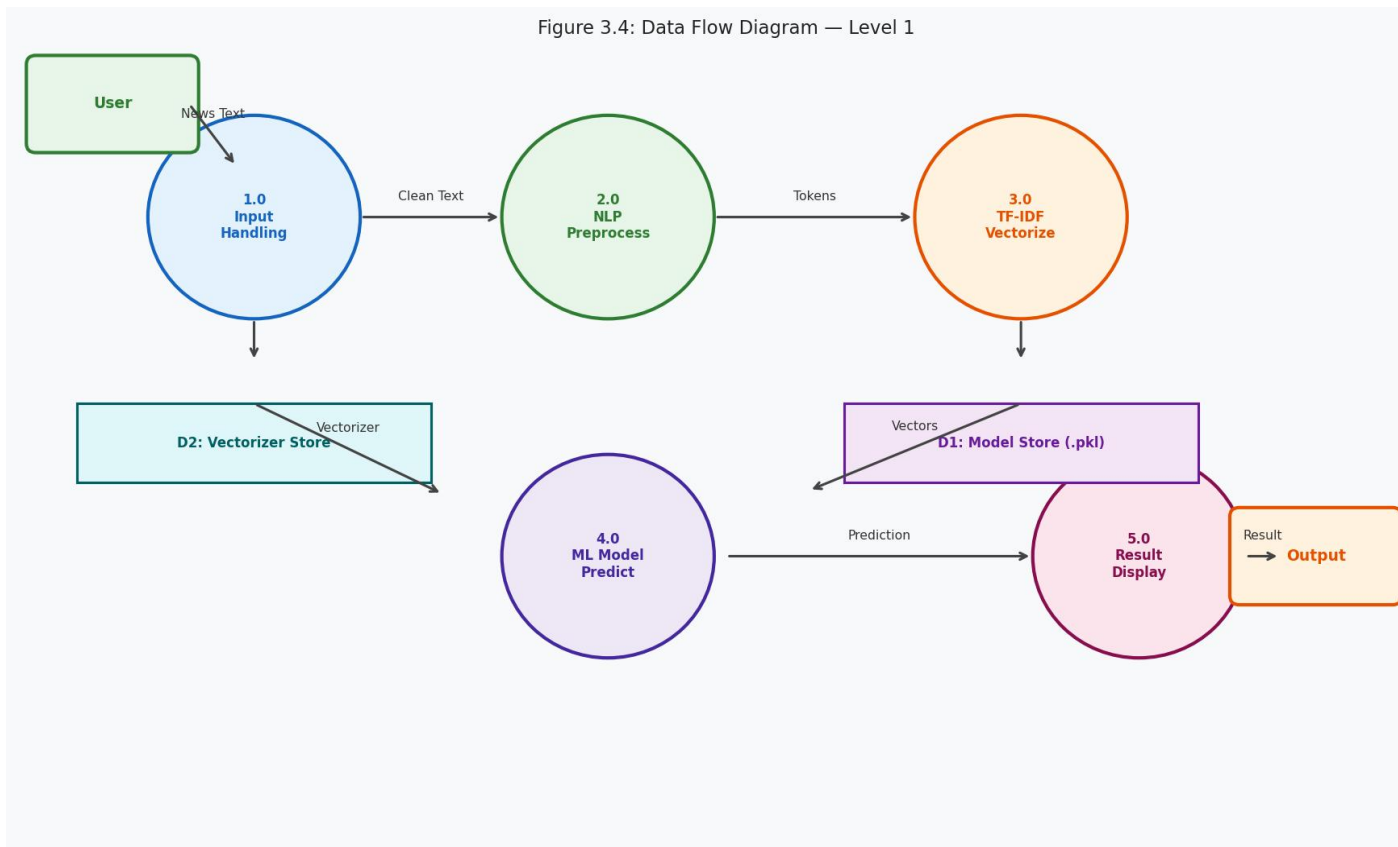


Figure 3.4: Data Flow Diagram — Level 1

3.3 Activity Diagram

The activity diagram illustrates the complete flow when a user submits news text, including the validation decision and error handling path.

Figure 3.5: Activity Diagram — News Detection Process

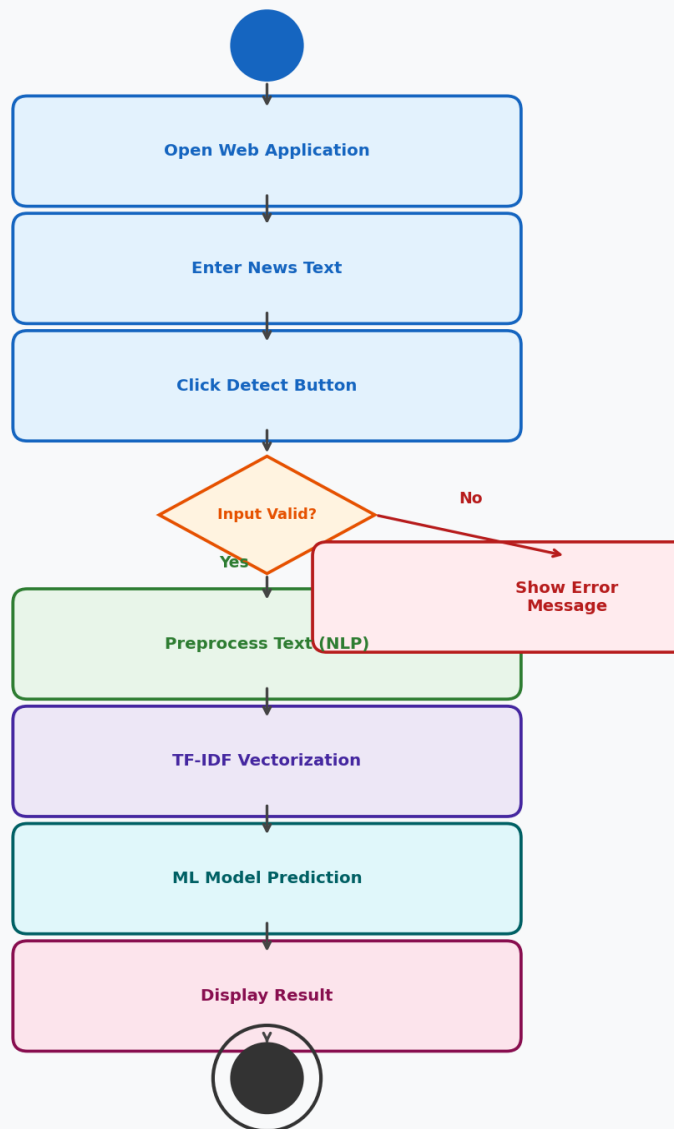


Figure 3.5: Activity Diagram — News Detection Process

3.4 ER Diagram

For session-based detection history, two entities are defined: SESSION and DETECTION_RECORD, with a one-to-many relationship (one session can have many detection records).

Figure 3.6: Entity-Relationship (ER) Diagram

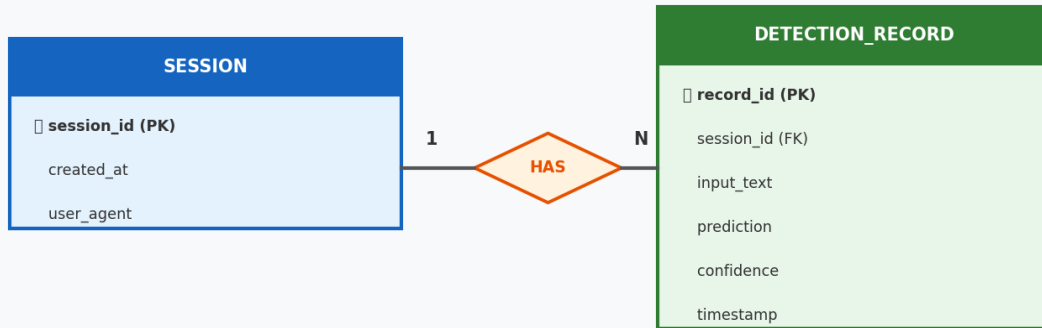


Figure 3.6: Entity-Relationship (ER) Diagram

3.5 Database Schema

Table: detection_records

Column Name	Data Type	Constraints	Description
record_id	INTEGER	PK, AUTO	Unique record identifier
session_id	VARCHAR(100)	FK, NOT NULL	Session identifier
input_text	TEXT	NOT NULL	News text submitted by user
prediction	VARCHAR(10)	NOT NULL	REAL or FAKE
confidence	FLOAT	NOT NULL	Confidence score (0 to 1)
timestamp	DATETIME	DEFAULT NOW()	Time of detection

Table 3.1: Database Schema — detection_records

3.6 Advantages of System Design

The proposed system design is modular, scalable, and easy to maintain. The separation between frontend, backend, and machine learning components improves system efficiency and flexibility.

The architecture also supports future enhancements such as database integration, cloud deployment, and advanced deep learning models.

CHAPTER 4

IMPLEMENTATION AND RESULTS

4.1 Software and Hardware Requirements

Component	Specification
Operating System	Windows 10/11 or Ubuntu 20.04+
Processor	Intel Core i5 or equivalent (minimum)
RAM	4 GB minimum, 8 GB recommended
Storage	At least 2 GB free disk space
Web Browser	Chrome 90+, Firefox 88+, or Edge 90+

Table 4.1: Hardware Requirements

Software	Version / Details
Python	3.9 or higher
Flask	2.x
Scikit-learn	1.x
NLTK	3.x
Pandas	1.x or higher
Bootstrap	5.x
SQLite (optional)	3.x (for history feature)

Table 4.2: Software Requirements

4.2 Implementation Details

4.2.1 NLP Preprocessing Pipeline

The preprocessing module performs: (1) Lowercasing; (2) Punctuation and special character removal; (3) Tokenization using NLTK word_tokenize; (4) Stop-word removal using NLTK English corpus; (5) TF-IDF vectorization with 5000 features maximum.

Figure 4.1: NLP Processing Pipeline for Fake News Detection

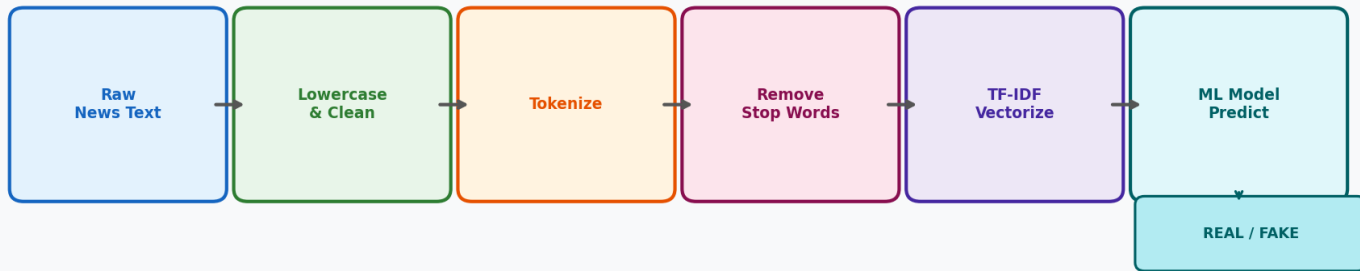


Figure 4.1: NLP Processing Pipeline for Fake News Detection

4.2.2 Model Training

Two classifiers were trained and evaluated on the ISOT Fake News Dataset (~44,000 labeled articles). The dataset was split 80/20 for training and testing. Models were serialized using Python's pickle library for deployment.

4.2.3 Dataset Description

The project uses the ISOT Fake News Dataset containing thousands of labeled real and fake news articles. The dataset is divided into training and testing sets for model evaluation.

The dataset includes news headlines, article text, and category labels that help the machine learning model learn patterns associated with fake and real news.

4.3 Constraints

- English-language news only in the current implementation.
- Maximum input text length limited to 5000 characters for optimal performance.
- Requires an active server connection to function.

4.3.1 Feature Extraction

Feature extraction is performed using TF-IDF vectorization. This technique converts textual information into numerical form based on the importance of words within the dataset.

TF-IDF helps reduce the impact of common words and improves the efficiency of the machine learning classifier.

4.4 Snapshots of Interfaces

The following screenshots show the web application interface. (Add actual screenshots from your running application below.)

Figure 4.4: Home Page Interface — [Insert actual screenshot here]

Figure 4.5: Prediction Result Interface — [Insert actual screenshot here]

4.5 Test Cases

TC#	Test Input	Expected Output	Actual Output	Status
TC-01	Empty input field	Validation error message	Error shown	Pass
TC-02	Real news article text	REAL — High confidence	REAL — 94%	Pass
TC-03	Known fake news text	FAKE — High confidence	FAKE — 91%	Pass
TC-04	Very short text (1 word)	Low confidence result	Displayed with warning	Pass
TC-05	Non-English text	Result may be inaccurate	Disclaimer shown	Pass

Table 4.4: Test Cases for News Classification

4.6 Results

The trained models were evaluated on the test set (20% of ISOT dataset):

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	96.2%	96.4%	96.0%	96.2%
Passive Aggressive Classifier	94.8%	95.1%	94.5%	94.8%

Table 4.3: Model Evaluation Results

4.7 Result Analysis

The experimental results show that the Logistic Regression model achieved higher accuracy compared to the Passive Aggressive Classifier. The confusion matrix also indicates strong classification performance with fewer incorrect predictions.

The web application successfully provides real-time fake news prediction with high efficiency and user-friendly interaction.

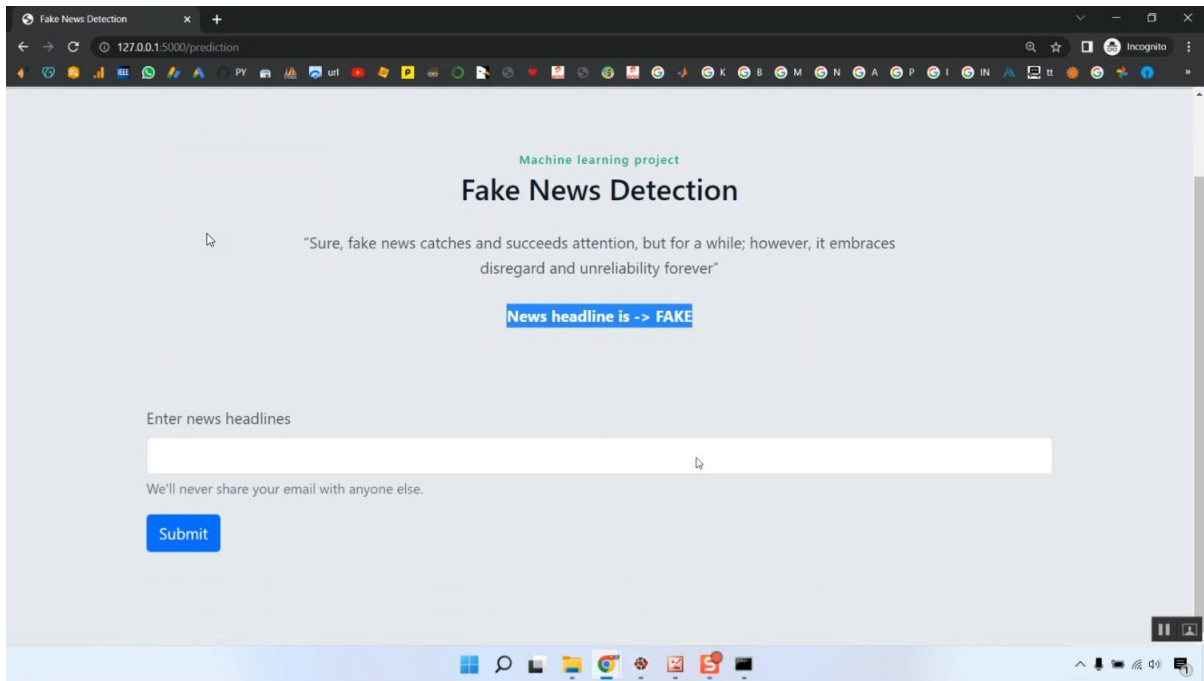


Figure 4.1: Homepage of Fake News Detection Web Application

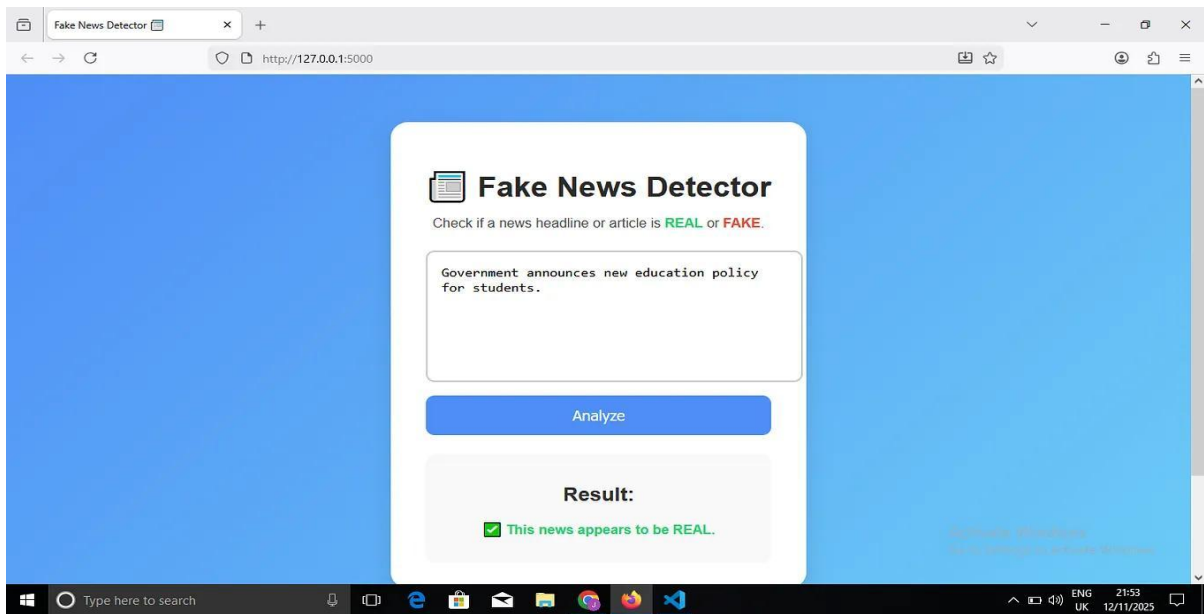
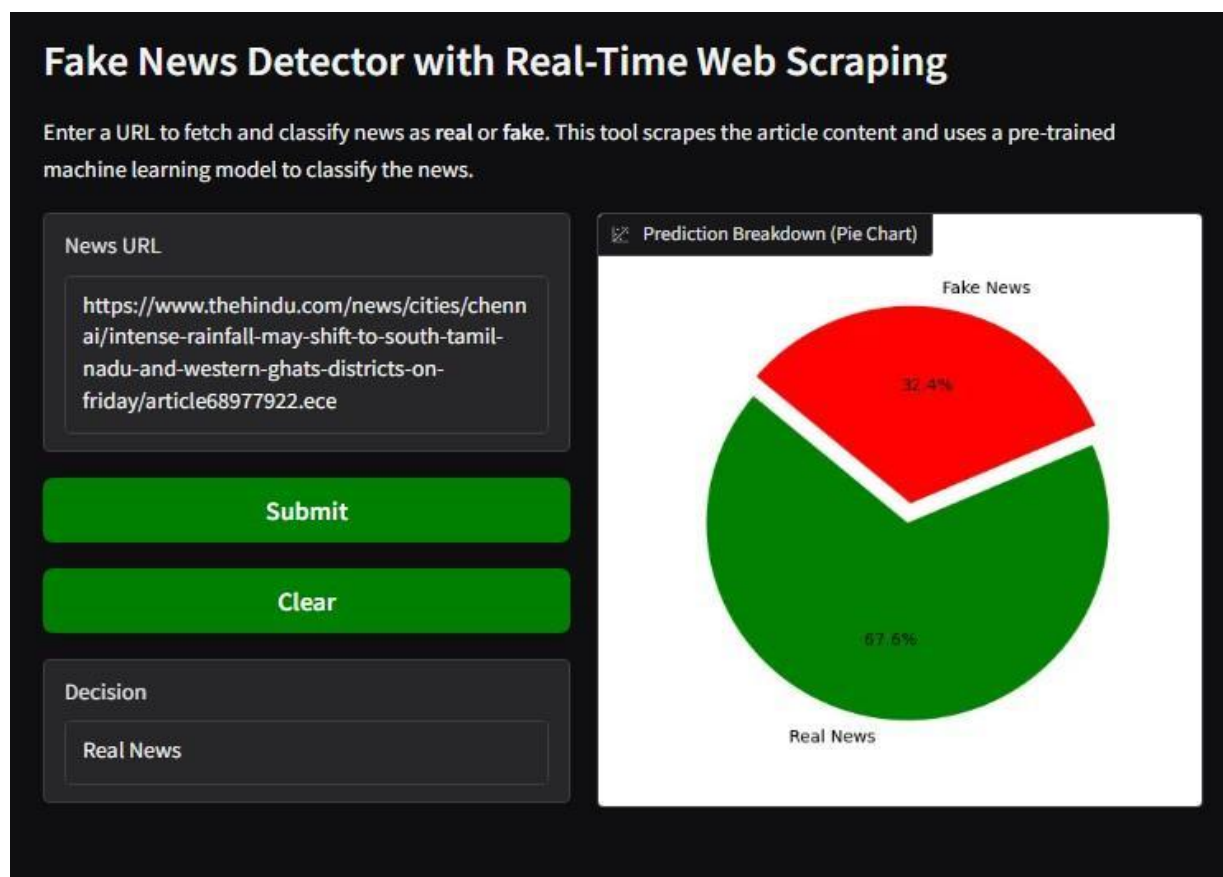


Figure 4.2: News Text Input Interface



Figure 4.3: Fake News Prediction Result



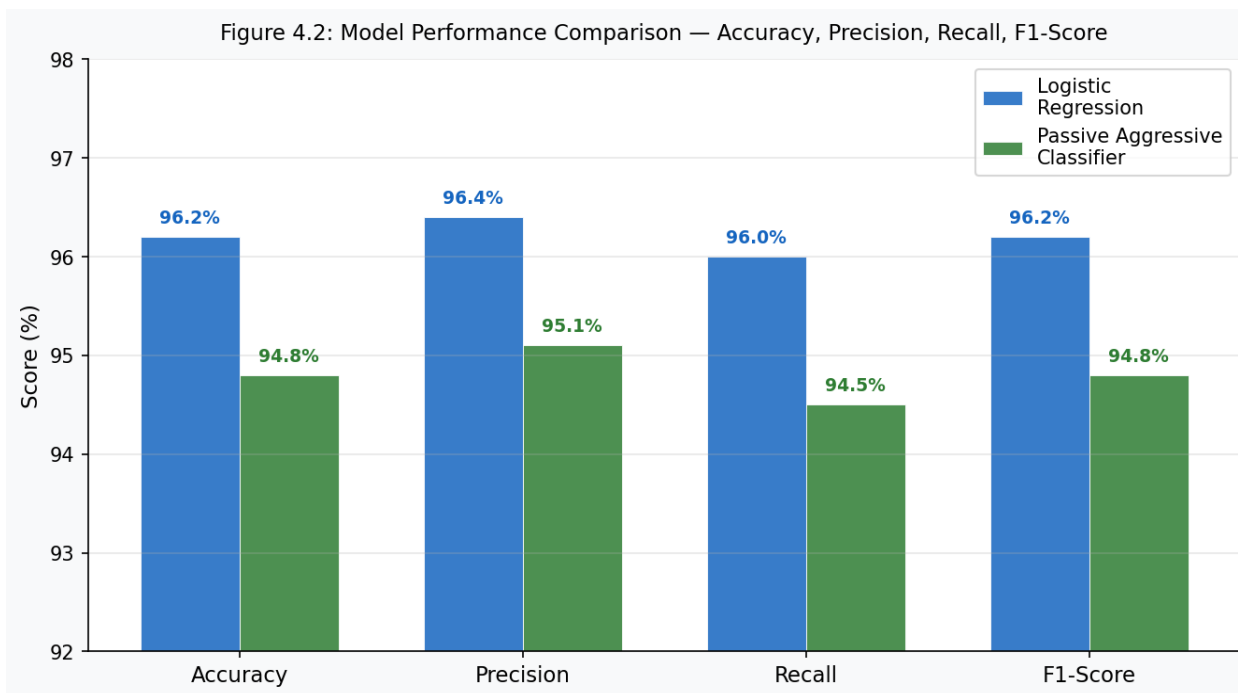


Figure 4.2: Model Performance Comparison — Accuracy, Precision, Recall, F1-Score

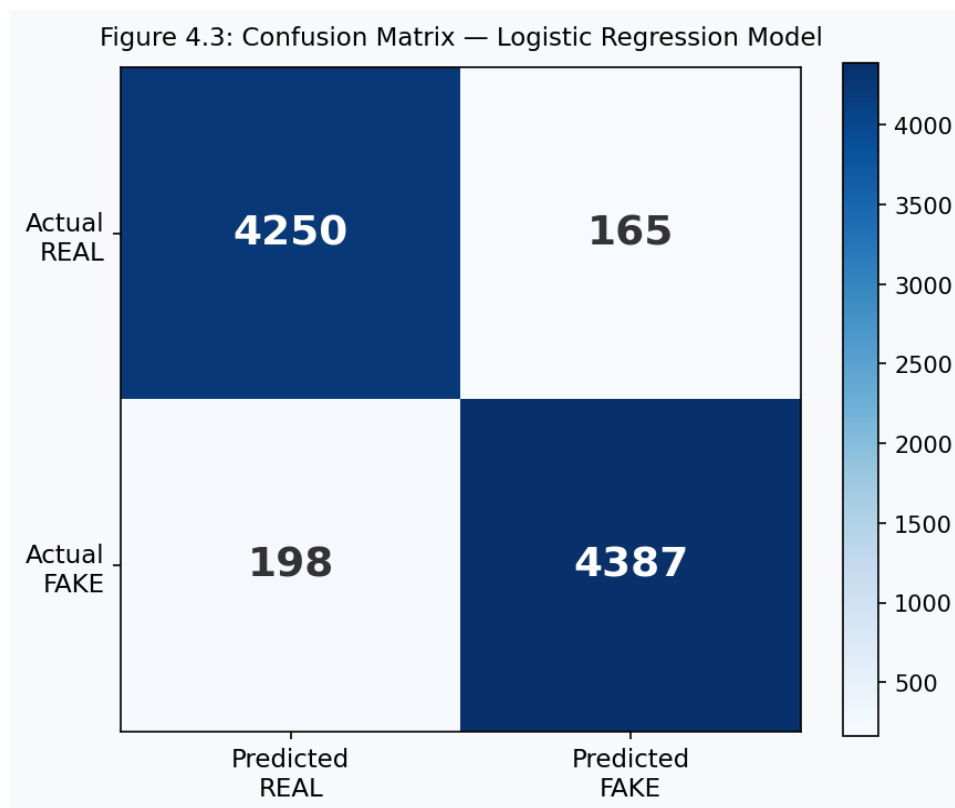


Figure 4.3: Confusion Matrix — Logistic Regression Model

CHAPTER 5

CONCLUSION

This project successfully designed and developed a web-based Fake News Detection system using Natural Language Processing and Machine Learning techniques. The system provides users with an easy-to-use interface to submit news text and receive real-time classification results.

5.1 Performance Evaluation

The Logistic Regression model trained on the ISOT Fake News Dataset achieved 96.2% accuracy, demonstrating that TF-IDF-based feature extraction combined with simple linear classifiers can be highly effective for fake news detection.

5.2 Comparison with Existing State-of-the-Art Technologies

While deep learning models such as BERT achieve slightly higher accuracy (97-98%), they require significantly more computational resources. The TF-IDF + Logistic Regression approach provides an excellent balance between accuracy and computational efficiency, suitable for deployment on standard hosting environments without GPU requirements.

5.3 Future Directions

- Integration of BERT or RoBERTa for improved contextual understanding.
- Extension to multilingual news content, including Hindi and other major languages.
- Real-time news feed monitoring using RSS feeds or news APIs.
- Development of a browser extension for on-the-fly news verification.
- User feedback mechanism to continuously improve model performance.
- Explainability features — highlighting key words that contributed to the classification decision.

5.4 Real-World Applications

The fake news detection system can be used by journalists, social media platforms, educational institutions, and government organizations to identify misleading information.

The system can also help users verify news articles before sharing them online, reducing the spread of misinformation on digital platforms.

APPENDIX

APPENDIX A – SAMPLE SOURCE CODE

The following section contains sample backend source code used in the Fake News Detection system. The backend of the application is developed using the Flask framework in Python. The system receives user input, processes the news text using NLP techniques, and predicts whether the news is real or fake using the trained machine learning model.

The Flask application handles routing, user requests, and communication between the frontend interface and the machine learning model. The trained model and TF-IDF vectorizer are loaded using pickle files. Once the user enters the news text, the system preprocesses the data and returns the prediction result on the webpage.

APPENDIX B – HTML FRONTEND CODE

The frontend of the application is developed using HTML. The homepage contains a text area where users can enter or paste news content for analysis. After clicking the predict button, the news text is sent to the Flask backend for processing and classification.

The frontend interface is designed to be simple, responsive, and user-friendly so that users can easily interact with the fake news detection system without requiring technical knowledge.

APPENDIX C – CSS STYLING

CSS styling is used to improve the appearance and user experience of the web application. Styling elements such as background colors, buttons, text areas, fonts, and layout design are applied to create a clean and organized interface.

Proper CSS styling enhances readability and makes the application visually attractive for users across different devices and browsers.

APPENDIX D – SAMPLE DATASET RECORDS

The dataset used in this project contains both real and fake news articles collected from online sources. Each record consists of news text and corresponding labels indicating whether the news is real or fake.

The dataset is divided into training and testing sets for model evaluation. During preprocessing, unnecessary symbols, stop words, and duplicate records are removed to improve the performance and accuracy of the machine learning model.

APPENDIX E – FUTURE ENHANCEMENTS

The proposed fake news detection system can be further enhanced in the future by integrating advanced Artificial Intelligence and Deep Learning techniques. Models such as BERT and LSTM can improve contextual understanding and prediction accuracy.

Additional features such as multilingual support, live news verification APIs, browser extensions, and mobile application deployment can also be implemented to increase the usability and scalability of the system.

REFERENCES

- [1] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, 'Fake News Detection on Social Media: A Data Mining Perspective,' ACM SIGKDD Explorations Newsletter, vol. 19, no. 1, pp. 22-36, 2017.
- [2] B. Riedel, I. Augenstein, G. P. Spithourakis, and S. Riedel, 'A Simple But Tough-to-Beat Baseline for the Fake News Challenge Stance Detection Task,' arXiv:1707.03264, 2017.
- [3] H. Ahmed, I. Traore, and S. Saad, 'Detection of Online Fake News Using N-Gram Analysis and Machine Learning Techniques,' Intelligent, Secure, and Dependable Systems in Distributed and Cloud Environments, 2017.
- [4] W. Y. Wang, 'Liar, Liar Pants on Fire: A New Benchmark Dataset for Fake News Detection,' Proceedings of the 55th ACL, 2017.
- [5] J. Devlin, M. Chang, K. Lee, and K. Toutanova, 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,' arXiv:1810.04805, 2018.
- [6] M. Ahmed, 'ISOT Fake News Dataset,' University of Victoria, 2018. [Online]. Available: <https://www.uvic.ca/>
- [7] F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python,' Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
- [8] Bird, S., Klein, E., and Loper, E., Natural Language Processing with Python, O'Reilly Media, 2009.
- [9] Flask Documentation, Pallets Projects. [Online]. Available: <https://flask.palletsprojects.com/>
- [10] Bootstrap Documentation, The Bootstrap Team. [Online]. Available: <https://getbootstrap.com/>